

One graph, one data model

The architecture beneath Onam — how seven clouds and every capability share one store, one graph, one source of truth.

Canonical finding

Per-tenant graph

Shared writer

Deterministic IDs

Rules as code

EXECUTIVE SUMMARY

Every capability writes to one place. That is the architecture.

Most security platforms are a drawer of tools that each keep their own findings and their own view of the world. Onam is built the opposite way: every engine, for every cloud, writes the same finding contract into one shared store, and everything reasons over one property graph. This paper explains that design and why it is the thing that makes cross-cloud attack paths, priced risk, and painless extensibility possible.

The architectural claim is specific and testable: there are **no per-engine report tables** to reconcile. A gateway reads only from one canonical findings store; a per-tenant graph turns those findings and the assets they touch into nodes and typed edges. Adding a capability does not add a silo — it adds another writer to the same store.

Why it matters to a buyer. Consolidation stops being a slide and becomes a data model.

Posture, identity, data, workloads, SaaS, and runtime detection share one schema, so a finding in one domain can form part of an attack path that ends in another — which is impossible when each tool hoards its own results.

01 Ingestion into a normalized inventory

Each cloud connects read-only and agentless (Whitepaper 01, §02). The pipeline runs four stages — **enumeration** → **enrichment** → **normalization** → **drift** — and writes two things every capability depends on: an asset inventory and an asset-relationship table.

FIGURE 1 — MANY CLOUDS, ONE NORMALIZED STORE



Seven clouds plus SaaS collapse into one normalized identity space, then into one findings store and one relationship table. Everything downstream reads from here.

02 The canonical finding contract

Consolidation lives or dies on the schema. Onam requires every engine to emit the **same standardized finding**, keyed consistently, so one table can hold results from posture, identity, data, workload, code, and SaaS engines without translation.

Field	Canonicalizes	Why it matters
resource_uid	ARN vs OCID vs self-link	One identity for an asset across clouds
account_id	account vs subscription vs project	Ownership normalized across providers
finding_id	deterministic hash	Same issue → same ID on every rescan; stable history
tenant_id	not-null isolation key	Every row is tenant-scoped by construction
provider_metadata	provider-specific detail	Nothing is lost; specifics travel with the finding

Because `finding_id` is deterministic, remediation tracking is reliable: a fixed issue auto-resolves on the next PASS and auto-closes after 30 days unseen, without a human reconciling IDs by hand.

03 The property graph

On top of the findings and relationships, Onam builds a **Neo4j property graph per tenant**. Assets are nodes; edges are typed — containment, attachment, IAM trust, and network reachability — and roughly 25 catalog-driven derivers add the abuse-edges an attacker would use. This is the substrate for attack-path reasoning (Whitepaper 01).

Per-tenant, by design. The graph is not a shared multi-tenant structure with a filter bolted on. Each tenant has its own graph, and a `tenant_id` filter is applied at the database on every query — one of three isolation layers described in Whitepaper 04.

04 The gateway reads one store

A single BFF (backend-for-frontend) gateway serves the product — on the order of 166 endpoints across roughly 60 routers behind a FastAPI service — and it reads **only** from the canonical findings store, never from per-engine tables. That constraint is what keeps the "one view" promise honest: there is no second source of truth to drift.

FIGURE 2 — WRITE MANY, READ ONE



Engines fan in through a shared writer; the gateway fans out from a single store. No per-engine report tables exist to reconcile.

05 Scale without a rebuild

Work parallelizes per account, service, and region, so a fifty-account organization scans in roughly the same wall-clock as one. Onam publishes scan SLO targets by resource count and API latency targets rather than customer benchmarks:

Resources	Scan SLO	API target
< 1,000	15 min	p95 ≤ 500 ms read · attack-graph traversal < 3 s · findings-to-console < 5 min
1k – 10k	60 min	
10k – 100k+	2 – 4 hr	

06 Extensibility — the real moat

The payoff of one data model is what it costs to add the next capability. A new engine does not get a new database, a new API surface, or a new UI silo. It writes the **same finding contract** into the **same store** through the shared writer, keyed by (tenant_id, account_id, rule_id, resource_uid). The gateway and graph pick it up unchanged.

- 1 Rules as code.** The platform loads and evaluates versioned YAML rules; it never hardcodes them. New rules auto-extend every framework they map to.
- 2 New engine, same contract.** A new capability emits the canonical finding; attack-path reasoning and FAIR pricing apply to it for free.
- 3 Custom without forking.** A Rule Builder lets teams add their own rules; custom frameworks map to rule IDs — no code change.

Consolidation, provable. Because every capability shares the schema, "one platform instead of a drawer of tools" is not a claim about the sales bundle — it is a claim about where the data lives. That is the difference this architecture buys.

07 Summary

Onam normalizes seven clouds plus SaaS into one identity space, one canonical findings store, and one per-tenant property graph. A single gateway reads only from that store, so there is no second source of truth. Deterministic finding IDs make history reliable; catalog-driven drivers turn relationships into attack-path edges; and new capabilities extend the platform by writing the same contract rather than adding a silo. The graph is table stakes; the shared data model beneath it is the architecture that makes cross-cloud paths and priced risk possible.